How We Write R

The code structure used in every script of this course: the R version of "Pythonic"

R's equivalent of PEP 8 is the **tidyverse style guide** (https://style.tidyverse.org). These are the rules we follow, as a quick reference.

1. If you know Python

Python	R (this course)
PEP 8	tidyverse style guide
`import pandas as pd`	`library(tidyverse)` at the top of the file
`x = 5`	`x <- 5` (keep = for function arguments)
`df.query(...).groupby(...).agg(...)`	`df |> filter(...) |> group_by(...) |> summarise(...)`
`pandas.DataFrame`	`tibble` (a modern data frame)
`pathlib.Path(__file__).parent`	`here::here()`
`requirements.txt` / `venv`	`renv.lock` / `renv`
`black` (formatter), `flake8` (linter)	`styler`, `lintr`
`assert x > 0`	`stopifnot(x > 0)`
`None` / `NaN`	`NULL` / `NA`
index starts at **0**	index starts at **1**
`snake_case`	`snake_case` (same!)

2. Anatomy of a script

Every `.R` file in this course has the same six parts, top to bottom.

1 Header — What the script does, its input, its output, how to run it

```
# Input : data/raw/x.csv   Output: data/processed/x_clean.csv
```

2 Packages — library() calls only. No install.packages() in scripts

```
library(tidyverse); library(here)
```

3 Import — Read from data/raw/ with here(). Never edit raw files

```
raw <- read_csv(here("data", "raw", "x.csv"))
```

4 Transform — Pipelines, one step per line, grouped into numbered sections

```
# ---- 2. Clean ----
clean <- raw |> select(-status) |> mutate(...)
```

5 Check — stopifnot(): stop loudly if the data are not what you expect

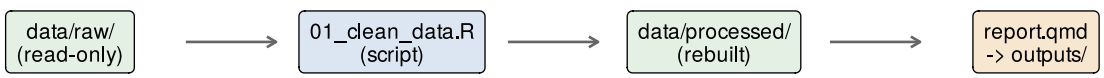
```
stopifnot("one row per country-year" = !any(duplicated(...)))
```

6 Save — Write to data/processed/ or outputs/ with code (write_csv, ggsave)

```
write_csv(clean, here("data", "processed", "x_clean.csv"))
```

3. How files connect: the pipeline

Data flows one way. Each arrow is code; nothing is done by hand.



4. The rules, with examples

Names: `snake_case` for everything. Objects are nouns, functions are verbs, script files are numbered in run order.

```
# Yes                                  # No
gdp_2020 <- ...                        GDP.2020 <- ...    gdp2020 <- ...
clean_data <- function(df) ...         CleanData <- function(df) ...
01_clean_data.R, 02_model.R            final_v2_REAL.R
```

Assignment and spacing: `<-` to store, spaces around operators and after commas, 2-space indent, lines under 80 characters.

```
# Yes                                  # No
x <- mean(life_exp, na.rm = TRUE)      x=mean(life_exp,na.rm=TRUE)
```

Pipes: one step per line, `|>` at the end of the line. Read it out loud as "and then".

```
# Yes                                    # No: nested, read inside-out
dev |>                                   summarise(group_by(filter(dev,
  filter(year == 2020) |>                  year == 2020), region),
  group_by(region) |>                      m = median(life_expectancy))
  summarise(m = median(life_expectancy))
```

Sections: split long scripts with numbered headers. RStudio turns them into a clickable outline (`Ctrl+Shift+O`; Mac: `Cmd+Shift+O`).

```
# ---- 3. Missing values -------------------------------------------------------
```

Paths: always `here()`. It starts at the folder that contains the `.Rproj`, so it works on every laptop and inside Quarto.

```
# Yes                                    # No
read_csv(here("data", "raw", "x.csv"))   setwd("C:/Users/Sara/Desktop")
                                         read_csv("C:/Users/Sara/x.csv")
```

Comments: explain *why*, not *what*. The code already says what.

```
# Yes: use 2020 only; pooling years would count each country 21 times
# No : filter year 2020
```

Grouping: every `group_by()` ends with `ungroup()` (or `.groups = "drop"`), or later steps silently run per group.

Randomness: call `set.seed()` once before anything random, so results repeat exactly.

Checks: state your assumptions as code. A silent pass means the data are what you think.

```
stopifnot("internet use is a %" = all(between(clean$internet_use, 0, 100)))
```

Packages: `library()` at the top for packages you use a lot; `pkg::function()` for a one-off, so readers see where it comes from (`scales::label_dollar()`).

Functions: copied the same code three times? Turn it into a function.

Outputs: figures with `ggsave()` and tables with `write_csv()`, never by right-click or copy-paste.

5. Let the tools do the formatting

- **Native pipe shortcut:** *Tools* ⟶ *Global Options* ⟶ *Code* ⟶ *Editing*, tick **Use native pipe operator,** `|>`, click **OK**. Now `Ctrl+Shift+M` (Mac: `Cmd+Shift+M`) types `|>`.
- **Reformat a selection:** select code, press `Ctrl+Shift+A` (Mac: `Cmd+Shift+A`).
- **Restyle a whole file** to the tidyverse guide: `install.packages("styler")` once, then `styler::style_file("path/to/script.R")`.
- **Find style problems** (like `flake8`): `install.packages("lintr")` once, then `lintr::lint("path/to/script.R")`.

6. The reproducibility test

1. *Session* ⟶ *Restart R* (`Ctrl+Shift+F10`).
2. Open the script and click **Source** (`Ctrl+Shift+S`), or open the `.qmd` and click **Render**.
3. It finishes without errors and gives the same results ⟶ the code is reproducible.